



Circuit Breakers, Caching & Reliability for Production Agents

AICB-P2T3 · Day 10 · Agent Production-Ready

Instructor

VinUniversity · Phase 2 · Track 3 · Week 5

HÃY SUY NGHĨ...

“Khi LLM provider timeout trong production, agent của bạn sẽ tự phục hồi hay làm sập cả workflow?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Mục tiêu & timeline 2 giờ
2. Failure Modes
3. Circuit Breaker & Fallback
4. Caching & Cost Budgeting
5. Observability & SLO
6. Lab: Reliability Engineering
7. Tổng kết

Mục tiêu & timeline 2 giờ

Tập trung vào reliability primitives: circuit breaker, fall-back, cache, metrics, chaos test.

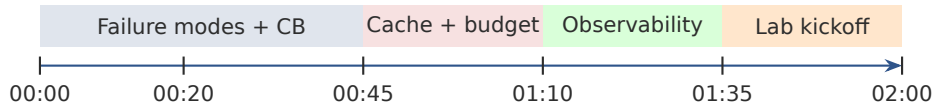
Conceptual outcomes

- Nhận diện 6 nhóm lỗi production của LLM agent.
- Giải thích 3 trạng thái circuit breaker.
- Phân biệt exact cache, semantic cache, tool-result cache.
- Thiết kế SLI/SLO/SLA cho agent.

Practical outcomes

- Xây gateway có fallback chain.
- Log latency/cost/cache hit/circuit state.
- Chạy chaos test và load test nhỏ.
- Viết report có metric để chấm điểm.

Trong lớp: hoàn thành baseline reliability harness. Bài lab mở rộng 4 giờ giúp phân loại nhóm hoàn thành sớm và nhóm đào sâu.



Lý thuyết có tương tác

- 20' reliability failure map
- 25' circuit breaker + fallback
- 25' caching + cost budget
- 25' metrics + chaos thinking

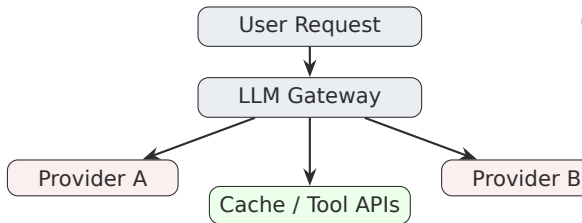
Lab kickoff trong lớp

- 10' repo walkthrough
- 15' team planning + first run
- Sau lớp: hoàn thiện 4h lab/report

Failure Modes

Reliability bắt đầu từ việc gọi đúng tên lỗi: transient, outage, degraded, stale, costly, unsafe.

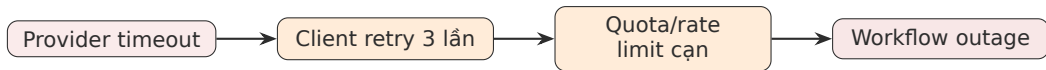
02



Lỗi có thể xuất hiện ở provider, gateway, cache, tool, hoặc business action. Reliability là **system property**, không phải chỉ là thêm retry.

6 loại lỗi cần monitor

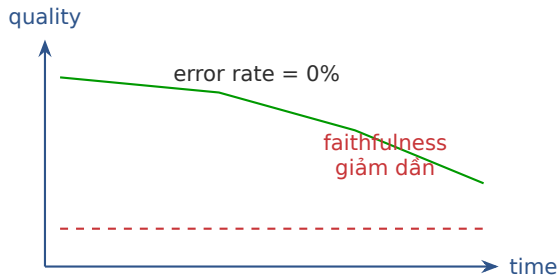
1. Provider transient: 429/500/timeout.
2. Degraded latency: P95 tăng mạnh.
3. Full outage: provider không phản hồi.
4. Orchestration loop: state/retry sai.
5. Tool/cache failure: stale/schema/auth.
6. Business action sai: side effect không rollback.



Think-pair-share: 5 phút

Hãy chọn một sản phẩm agent bạn biết. Nếu provider chính bị timeout 30 giây, người dùng sẽ thấy gì? Nhóm đề xuất một cách **contain, isolate, recover**.

Retry chỉ là bước đầu. Nếu không có circuit breaker + fallback + budget, retry có thể biến lỗi nhỏ thành outage lớn.



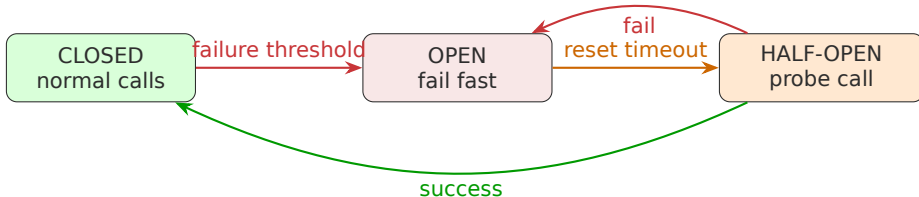
Nguyên nhân thường gặp

- Provider cập nhật model silently.
- Prompt/schema thay đổi nhưng eval không đổi.
- Knowledge base stale hoặc retrieval yếu.
- Cache trả câu đúng cũ nhưng sai hiện tại.

Lưu ý: Quality SLO phải đi cùng uptime SLO. Error rate = 0% không đủ.

Circuit Breaker & Fallback

Circuit breaker ngắt gọi provider đang hỏng; fallback chain giữ trải nghiệm user ở mức chấp nhận được.



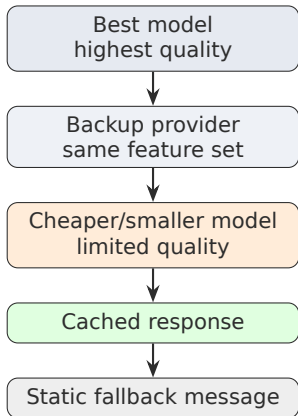
Nên có breaker theo từng provider/model/task. Provider A open không nên kéo provider B hoặc cache layer sập theo.

```
class CircuitBreaker:
    def call(self, fn, *args, **kwargs):
        if self.state == "OPEN":
            if not self.ready_to_probe():
                raise CircuitOpenError()
            self.state = "HALF_OPEN"
        try:
            result = fn(*args, **kwargs)
            self.record_success()
            return result
        except Exception:
            self.record_failure()
            raise
```

Các tham số chính

- failure_threshold
- reset_timeout_seconds
- success_threshold
- exception nào được tính là failure

Lưu ý: Production multi-instance: state nên có backend chung như Redis, không chỉ in-memory.



Fallback không chỉ là đổi model.
Cần kiểm tra **feature compatibility**: JSON mode, tool calling, context length, latency/cost, policy behavior.

Nhóm 3 người - 8 phút

Mỗi nhóm chọn 1 task: customer support, code review, medical triage, hoặc internal HR chatbot. Thiết kế fallback ladder 4 bậc và nêu task nào **không được phép** fallback sang model yếu hơn.

Gợi ý trade-off

- Quality vs latency
- Cost vs safety
- Cached answer vs freshness
- Static response vs user trust

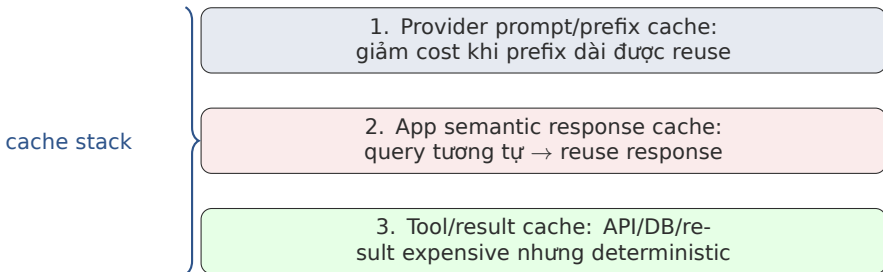
Output cần nộp

- Ladder 4 bậc
- Điều kiện chuyển bậc
- Metric để kiểm chứng
- Rủi ro lớn nhất

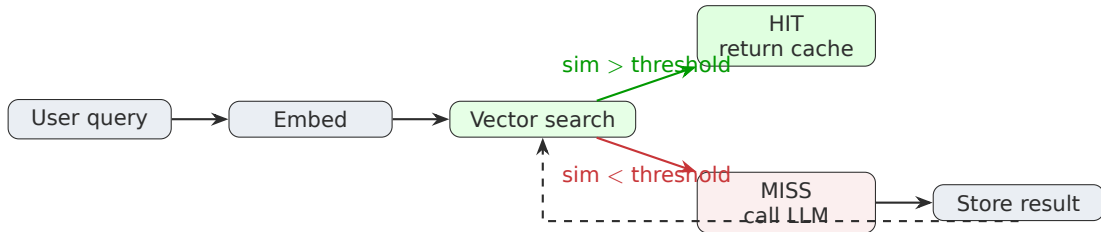
Caching & Cost Budgeting

Cache đúng chỗ có thể giảm latency/cost; cache sai chỗ tạo stale answer và hallucination ổn định.





Cache deterministic và low-risk trước. Với semantic response cache, cần threshold, TTL, invalidation, và allowlist theo task.



Lưu ý: Cache poisoning: hai query cosine gần nhau nhưng intent khác nhau. Metric quan trọng: hit rate **và** false-hit rate.

3 lớp control

1. Per-request cap: max tokens, max tools, timeout.
2. Per-user/app rate limit: token bucket.
3. Monthly budget: warn 80%, hard stop/route cheap at 100%.

Metric cần log

- provider, model, route reason
- input/output tokens, estimated cost
- cache hit/miss, similarity score
- latency, status, circuit state

Đừng chỉ tổng cost theo ngày. Cần cost theo feature/user/model để tìm đường call đắt và tối ưu đúng nơi.

Use case	Cache decision	Rủ ro chính
FAQ admissions	nên cache semantic	thông tin deadline stale
Account balance	không cache response	privacy + freshness
Code explanation	cache có điều kiện	context khác nhau
Weather today	tool cache TTL ngắn	stale theo thời gian
Policy summary	cache + event invalidation	policy update

Vote nhanh - 5 phút

Với mỗi dòng, giơ tay: cache / không cache / cache có điều kiện. Giảng viên chọn 2 ý kiến trái chiều để tranh luận.

Observability & SLO

Không đo thì không biết system đang tốt, chậm, đắt, hay đang trả lời sai.

Khái niệm	Ý nghĩa	Ví dụ trong lab
SLI	metric đo được	availability, P95 latency, cache hit rate, false-hit rate
SLO	target nội bộ	availability $\geq 99\%$, P95 $< 2.5s$, fallback success $\geq 95\%$
SLA	cam kết bên ngoài	99.5% uptime/tháng cho customer-facing API
Error budget	mức lỗi được phép	nếu burn rate cao $\rightarrow$ freeze feature, ưu tiên reliability

Lưu ý: LLM agent cần thêm quality SLO: faithfulness, safety pass rate, escalation correctness.

```
REQUESTS = Counter("agent_requests_total", [
    "provider", "status", "route"])
LATENCY = Histogram("agent_latency_seconds", [
    "provider", "route"])
CACHE_HITS = Counter("cache_hits_total", [
    "cache_type"])
CIRCUIT_STATE = Gauge("circuit_state", [
    "provider"]) # 0 closed, 1 open, 2 half-open
```

Report cần có

- Latency P50/P95/P99.
- Availability/error rate.
- Fallback success rate.
- Cache hit rate và false-hit examples.
- Recovery time trong chaos test.

Chaos scenarios trong lab

1. Primary provider timeout 100%.
2. Primary provider intermittent 50%.
3. Cache returns stale candidate.
4. Cost cap gần cạn.

Expected evidence

- Circuit chuyển CLOSED → OPEN.
- Gateway route sang fallback.
- Không retry storm.
- Metrics/report ghi rõ recovery time.

Mini design review - 7 phút

Mỗi nhóm viết 1 chaos scenario mới và metric chứng minh system recover. Nhóm khác phản biện: scenario đó có side effect không?

Lab: Reliability Engineering

Lab được thiết kế 4 giờ; trên lớp kickoff 2 giờ. Học viên giỏi có thể hoàn thành core sớm và làm stretch tasks.

Mục tiêu: Build reliability gateway: circuit breaker + semantic/tool cache + metrics + chaos report

Deliverable: Repo hoàn chỉnh, metrics JSON/CSV, report Markdown/PDF, demo command chạy được

Thời gian: 2 giờ trên lớp + 2 giờ mở rộng

Thời gian	Việc cần làm	Deliverable
0-30'	Setup repo, chạy tests baseline, đọc TODO	screenshot/test log
30-75'	Implement circuit breaker + fallback router	state transition log
75-120'	Implement metrics + run mini chaos test	metrics.json lần 1
120-180'	Implement cache + TTL/threshold tuning	cache comparison table
180-240'	Load test + report + rubric self-check	final report + plots/CSV

Core pass khoảng 2 giờ cho nhóm mạnh. Stretch 2 giờ còn lại: false-hit analysis, cost simulation, report chất lượng cao, và test coverage.

Metrics bắt buộc

- availability, error rate
- latency P50/P95/P99
- fallback success rate
- circuit open count + recovery time
- cache hit rate + estimated cost saved
- chaos scenario pass/fail

Report bắt buộc

- architecture diagram ngắn
- config table
- experiment setup
- metrics table trước/sau cache
- failure analysis
- next steps

Lưu ý: Report không có metric định lượng = không đạt phần grading.

Hạng mục	Điểm	Kỳ vọng
Circuit breaker/fallback	25	state machine đúng, không retry storm, fallback có route reason
Cache/cost	20	hit rate/cost saved rõ, TTL/threshold có giải thích, false-hit examples
Observability/metrics	20	P50/P95/P99, availability, circuit state, cache metrics reproducible
Chaos/load test	20	ít nhất 3 scenarios, có recovery evidence
Report/code quality	15	README rõ, tests, type hints, config, report dễ chấm

1. Chạy một command tạo metrics: `make run-chaos` hoặc tương đương.
2. Chỉ ra breaker chuyển state khi primary fail.
3. So sánh latency/cost có cache và không cache.
4. Mở report và giải thích 1 failure mode còn tồn tại.
5. Nêu 1 config bạn sẽ đổi nếu deploy production.

Tổng kết

Reliability engineering giúp agent fail gracefully, đo được, và có thể cải thiện bằng dữ liệu.

07

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Circuit breaker + fallback là minimum viable reliability cho agent production.
- 2 Cache có ROI cao nhưng cần guardrail: TTL, threshold, invalidation, false-hit tracking.
- 3 Metrics phải bao phủ latency, availability, cost, cache, circuit state và quality.
- 4 Chaos/load test biến giả định thành bằng chứng; report định lượng giúp chấm điểm công bằng.

1. Microsoft Azure Architecture Center: Circuit Breaker pattern.
2. Release It! Design and Deploy Production-Ready Software, Michael Nygard.
3. Prometheus client Python: Counter, Gauge, Histogram docs.
4. LiteLLM documentation: routing, retries, fallbacks.
5. Langfuse documentation: LLM observability, traces, cost and latency metrics.

Hỏi & Đáp
